TESTING EN AMBIENTES ÁGILES

1. Punto de partida: Agile frente a cascada

La presentación abre con diagramas que contrastan el trabajo secuencial del enfoque tradicional («waterfall» o cascada) con la dinámica iterativa del software ágil (diap. 2, 11-13). No se plantea que en Agile desaparezcan requisitos, diseño, implementación o pruebas, sino que estas actividades se recorren y revisitan en ciclos cortos.

Tradicional / cascada	Requisitos → diseño → desarrollo → testing/verificación → despliegue. El avance suele representarse como una sucesión de etapas. Una falla descubierta al probar al final puede obligar a regresar a decisiones anteriores.
Ágil / iterativo	Se planifican, diseñan, desarrollan, prueban y entregan porciones pequeñas de producto. El feedback llega dentro de la iteración y puede modificar lo siguiente que se hace.

La diapositiva 13 ilustra explícitamente que, dentro de la iteración ágil, coexisten nueva funcionalidad, corrección de defectos y testing; el modelo secuencial mostrado a la derecha ordena requirements, design, coding, testing y release. La lámina 11 también compara el ciclo de vida del testing con el ciclo Agile: ambos tienen actividades propias, pero el testing debe acompañar las entregas.

> Idea clave de clase. Agile no significa «trabajar sin pruebas»; significa probar continuamente, aprender temprano y ajustar el trabajo con ciclos de feedback más cortos.

2. Manifiesto del testing: los cinco cambios de mirada

El manifiesto visual ocupa las diapositivas 5 a 10. Cada contraste dice qué valor se prioriza en un equipo ágil; no debe interpretarse como que la segunda actividad queda prohibida.

2.1. Probar durante el proceso, no solamente al final (diap. 5-6)

La lámina afirma «Testing is an activity» en oposición a «Testing is a phase». Probar es una actividad transversal: desde entender criterios de aceptación hasta verificar el código y la entrega. Si se espera al cierre del proyecto, los defectos y malentendidos se conocen demasiado tarde.

- Ejemplo complementario: al implementar el registro de usuarios se discuten antes casos como correo inválido, contraseña vacía y correo ya existente; después se prueban en la misma iteración.

2.2. Prevenir defectos antes que limitarse a encontrarlos (diap. 5 y 7)

La presentación contrasta «Prevent bugs» con «Finding bugs». Además de detectar errores, el equipo evita introducirlos: define mejor lo que se espera, hace revisiones y verifica pronto supuestos ambiguos. Encontrar defectos sigue siendo útil; prevenirlos reduce el retrabajo.

2.3. Entender lo que se prueba, no sólo chequearlo (diap. 5 y 8)

La oposición entre «Are you a checker?» y «Be a tester!» señala que ejecutar una lista de controles no alcanza. El pensamiento crítico pregunta qué riesgo existe, qué escenario falta, qué comportamiento sería sorprendente y qué necesita realmente la persona usuaria.

- Ejemplo complementario: confirmar que un botón «Pagar» responde es un chequeo; investigar doble clic, pago rechazado, conexión interrumpida y duplicación del pedido es un análisis de comportamiento.

2.4. Ayudar a construir el mejor sistema, no enfocarse sólo en romperlo (diap. 5 y 9)

La prueba no es una competencia para demostrar que los desarrolladores «se equivocaron». El objetivo es aportar información útil para mejorar el producto, cuestionar supuestos y colaborar con quienes lo crean.

2.5. Responsabilidad colectiva por la calidad (diap. 5 y 10)

«Whole team takes responsibility for quality» contrasta con «Tester is responsible for quality». El tester desempeña un rol especializado, pero la calidad depende también de desarrollo, producto y las demás personas del equipo. No se delega toda la calidad a una «puerta de control» final.

> Idea clave de clase. Los cinco valores, en forma breve: durante el desarrollo; prevenir; comprender; construir mejor; calidad como responsabilidad de todos.

3. Nueve principios del testing en proyectos ágiles (diap. 14)

La presentación enumera nueve principios. A continuación se mantiene su formulación y se desarrolla qué implica cada uno en la práctica.

1. El testing se mueve hacia adelante en el proyecto

Se empieza a pensar en riesgos, ejemplos y verificaciones desde que surge una necesidad. El momento de descubrir una ambigüedad importante es antes de programar, no tras completar la historia.

2. El testing no es una fase

No hay que esperar un «turno» aislado para probar. Analizar criterios, diseñar pruebas, ejecutarlas y revisar los resultados forma parte del trabajo habitual.

3. Todos hacen testing

Desarrollo prueba su código; producto ayuda a aclarar resultados esperados; tester facilita estrategias y exploración. Las responsabilidades se complementan.

4. Reducir la latencia del feedback

Latencia = tiempo entre hacer un cambio y enterarse de si funciona. Un test automático rápido informa casi inmediatamente que se rompió una regla; descubrirlo días después encarece la corrección.

5. Las pruebas representan expectativas

Un caso de prueba registra una expectativa observable sobre el sistema. Bien definido, permite que todos conversen sobre qué significa «correcto».

6. Mantener el código limpio y corregir defectos rápido

No conviene dejar crecer una acumulación de problemas. Una base de código clara y correcciones prontas permiten seguir incorporando cambios con menor riesgo.

7. Reducir la sobrecarga de documentación de pruebas

Priorizar información útil y mantenible sobre documentación repetitiva. Esto no implica ausencia de evidencia o trazabilidad cuando es necesaria.

8. Las pruebas son parte del «done»

La tarea no está realmente terminada si el comportamiento no fue verificado según los acuerdos del equipo. «Done» es un criterio compartido, no simplemente «ya compila».

9. De probar al final a desarrollo conducido por pruebas

En lugar de programar todo y recién entonces testear, los ejemplos o pruebas pueden guiar la implementación desde el comienzo; la presentación luego destaca TDD y ATDD.

4. Seis prácticas concretas de testing ágil (diap. 15)

4.1. Pruebas unitarias y de integración automatizadas

Las pruebas unitarias verifican pequeñas unidades de comportamiento (por ejemplo, un método que calcula descuentos). Las pruebas de integración verifican el vínculo entre componentes o servicios (por ejemplo, lógica de pedidos y base de datos). La automatización permite repetirlas con cada modificación.

4.2. Pruebas de regresión automatizadas a nivel de sistema

Una regresión aparece cuando un cambio afecta algo que funcionaba. Repetir pruebas de flujos importantes permite detectar esos problemas. «A nivel de sistema» significa observar el comportamiento de componentes integrados, no necesariamente ejecutar todo por interfaz gráfica.

4.3. Pruebas exploratorias

El tester aprende sobre el producto mientras diseña y ejecuta nuevas pruebas. Utiliza conocimiento del negocio y observaciones para explorar escenarios no previstos en una lista fija. Es especialmente útil frente a incertidumbres, interfaces cambiantes y riesgos poco conocidos.

4.4. TDD (Test-Driven Development)

TDD es desarrollo guiado por pruebas. Explicación complementaria: suele describirse como el ciclo rojo → verde → refactorizar: escribir primero una prueba que falla, implementar lo mínimo para hacerla pasar y mejorar la estructura del código manteniendo las pruebas verdes. El foco inmediato suele estar en pequeñas unidades de comportamiento.

4.5. ATDD (Acceptance Test-Driven Development)

ATDD es desarrollo conducido por pruebas de aceptación. Se aclara qué espera el negocio de una historia y se acuerdan ejemplos verificables antes de terminar su implementación. La diferencia de énfasis respecto de TDD es que ATDD parte de las expectativas de aceptación y TDD suele guiar el diseño del código.

4.6. Control de versiones de pruebas y código

Guardar pruebas junto con el código permite revisar, compartir y modificar las verificaciones cuando evoluciona la funcionalidad. Ejemplo complementario: una rama de Git que modifica descuentos también incluye los tests que validan las nuevas reglas.

5. ATDD paso a paso: lectura del diagrama (diap. 16)

La diapositiva representa un circuito alrededor de una historia de usuario. Se observan nodos como «Característica / Feature (con ejemplos)», «Historia de Usuario», «Exploración de Ejemplos», «Pruebas de Aceptación de alto nivel», «Más Pruebas», «Automatización de las pruebas», «Programar y ejecutar las pruebas», «Otras Pruebas», «Explorar / Pruebas exploratorias», «Aceptación de la Historia» y «Corrección de Defectos».

- Arranque: la característica o historia se conversa mediante ejemplos, no solamente con una descripción breve.
- Definición de aceptación: esos ejemplos se transforman en condiciones o pruebas de aceptación de alto nivel.
- Implementación y verificación: se añaden más pruebas, se automatizan las convenientes y se programa/ejecuta para obtener feedback.
- Exploración y reparación: las pruebas exploratorias y otras pruebas detectan vacíos; los defectos vuelven al circuito de corrección.
- Cierre: la historia se acepta cuando satisface los acuerdos establecidos.

Ejemplo complementario de ATDD: historia «Como cliente quiero aplicar un cupón». Antes de implementarla, producto, tester y desarrollo acuerdan tres ejemplos: cupón válido reduce el total; cupón vencido no aplica; cupón de otra categoría es rechazado. Esos ejemplos orientan el trabajo y permiten comprobar cuándo la historia está lista.

6. El rol del tester dentro del equipo ágil (diap. 17)

El esquema sitúa al tester en el centro y lo conecta con developers, product owner y system team. La lámina no propone un tester aislado: muestra colaboración y asistencia bidireccional.

Con Product Owner / producto

- Contribuye a la preparación y revisión de historias y a la definición de criterios de aceptación.
- Aporta conocimiento del negocio e identifica dependencias entre historias o funcionalidades.

Con desarrolladores

- Ayuda a definir estrategias de prueba y promover automatización, incluida la integración continua según la lámina.
- Detecta y comunica bugs, aporta feedback temprano y sugiere mejoras de funcionalidad y usabilidad.

Con el equipo de sistema

- Participa en documentación de historias para preparar pruebas y en definición de API, casos y datos de pruebas de integración.
- Ayuda a conocer y comprender el comportamiento de las características; recibe y comunica información con producto y desarrolladores.

Tareas propias señaladas alrededor del tester

- Preparar y mantener ambientes de prueba.
- Definir casos y datos de prueba para las historias del sprint; verificar criterios y registrar/notificar bugs.
- Participar en la aceptación de la historia con el Product Owner y en el seguimiento de métricas de calidad.

> Idea clave de clase. El valor del tester no reside sólo en ejecutar casos: conecta expectativas del negocio, riesgos técnicos y evidencia real sobre la calidad.

7. Desafíos culturales del testing ágil (diap. 18)

El mapa mental de la presentación identifica cinco grupos de desafíos: el cambio no sucede fácilmente, cultura organizacional, expectativas de dirección, introducción del cambio y barreras para el éxito. Esta sección explica cómo interpretar los elementos visibles del diagrama.

El cambio no llega de manera automática

El diagrama propone paciencia, aprender de experiencias difíciles, construir credibilidad, desarrollo profesional, prevenir una mentalidad de «policía de calidad» y demostrar con acciones. La transición es un cambio en hábitos y colaboración, no sólo en nombres de reuniones.

La cultura organizacional importa

Aparecen filosofía de calidad, ritmo sostenible, relaciones con clientes, tamaño de la organización y empoderamiento del equipo. Todos influyen en la posibilidad de sostener un testing temprano y compartido.

La gestión y las expectativas deben adaptarse

El mapa menciona el cambio cultural de los managers y la necesidad de hablar su lenguaje. Explicación complementaria: conviene mostrar avances mediante riesgos reducidos, feedback y mejoras observables, no únicamente cantidad de casos ejecutados.

Barreras a superar

Se enumeran pérdida de identidad, roles adicionales, falta de formación, desconocimiento de conceptos ágiles, malas experiencias y diferencias culturales entre roles. No se resuelven automáticamente por adoptar una metodología.

Introducir el cambio

Hablar de temores, dar al equipo responsabilidad real y celebrar éxitos son acciones que el material asocia con una transición más saludable.

8. Cuadrantes del testing ágil (diap. 19)

La matriz cruza dos ejes: orientación al negocio vs. orientación a la tecnología (vertical) y pruebas que guían el desarrollo vs. pruebas que critican el producto (horizontal). Sirve para recordar que no todas las pruebas cumplen la misma función.

Q1 · Tecnología + guiar desarrollo	Pruebas unitarias y pruebas de componentes a nivel de código. Ayudan a construir correctamente las piezas.
Q2 · Negocio + guiar desarrollo	Ejemplos, aceptación de historias, experiencia de usuario (UX), prototipos y simulaciones. Ayudan a acordar qué producto construir.
Q3 · Negocio + criticar producto	Pruebas exploratorias, flujos de trabajo, usabilidad, aceptación de usuario (UAT), monitoreo y observabilidad. Buscan conocer la experiencia y descubrir comportamientos problemáticos.
Q4 · Tecnología + criticar producto	Rendimiento/carga, seguridad, recuperabilidad y otros atributos de calidad. Cuestionan si el producto soporta condiciones exigentes.

Cómo estudiarlos: Q1 y Q2 aportan feedback para orientar la construcción; Q3 y Q4 observan y desafían un producto ya disponible. La matriz no establece un orden temporal obligatorio. Ejemplo complementario: en un carrito de compras, Q1 prueba cálculo de totales, Q2 define ejemplos de descuentos, Q3 explora la experiencia real de compra y Q4 evalúa carga simultánea.

9. Pirámide del testing ágil (diap. 20-22)

La pirámide expresa una preferencia por tener muchas pruebas pequeñas y rápidas en la base y menos pruebas complejas y frágiles en la punta. En la diapositiva 20 se distinguen tres niveles y una nube externa para pruebas manuales/exploratorias.

Base: pruebas unitarias y de componentes

Validan reglas acotadas y suelen ejecutarse rápido. La base del diagrama aparece como «pruebas realizadas por el programador». Ejemplo complementario: un método que calcula intereses, con entradas y resultados esperados.

Nivel medio: capa de servicio o API

Incluye reglas de negocio y pruebas funcionales. Valida interacciones o comportamientos sin recorrer necesariamente toda la interfaz de usuario. Ejemplo complementario: enviar una solicitud a una API de pedidos y verificar estado y total.

Punta: flujos mediante la interfaz gráfica

Comprueban recorridos completos del usuario y dependen de más piezas. Son importantes pero su mantenimiento puede costar más. Ejemplo complementario: iniciar sesión, agregar productos y confirmar una compra desde un navegador.

Pruebas manuales y exploratorias

En el esquema aparecen por encima de la pirámide: permiten descubrir comportamientos que pruebas guionadas o automatizadas no anticiparon. No son «innecesarias» por no estar en la base.

Segunda visualización: niveles de pruebas (diap. 21)

La variante mostrada distingue pruebas unitarias, aceptación, pruebas de sistema y restricciones, exploratorias y «otras». Anota que las unitarias favorecen el desarrollo, aceptación y regresión ayudan a encontrar huecos, y que algunas pruebas se ejecutan automáticamente mientras que otras se ejecutan manualmente o no resulta práctico automatizarlas.

Comparación tradicional / ágil y sus proporciones (diap. 22)

La lámina representa la pirámide tradicional invertida, asociada a buscar bugs, y la ágil con base ancha, asociada a prevenir bugs. Los valores ilustrativos escritos en la imagen son:

Tradicional (gráfico)	Pruebas GUI end-to-end: 80-90%; integración: 5-15%; unitarias: 1-5%.
Ágil (gráfico)	Pruebas unitarias: 80-90%; aceptación: 5-15%; GUI: 1-5%.

Precisión importante: son proporciones de referencia de ese gráfico, no cuotas universales obligatorias. La combinación real depende del contexto, el riesgo y la arquitectura.

> Idea clave de clase. Diferenciar «cuadrantes» y «pirámide»: los cuadrantes clasifican el propósito y la perspectiva; la pirámide aconseja cómo equilibrar distintos niveles de automatización.

10. Niveles de prueba y capas del sistema (diap. 23)

El diagrama representa browser, web layer, business layer, data layer y database, con flechas que señalan unit, integration, system y acceptance. La idea es que cada nivel atraviesa una cantidad distinta de componentes del sistema.

- Unitarias: verifican piezas de código por separado, con dependencias controladas cuando corresponde.
- Integración: comprueban que distintos módulos, capas o servicios colaboren correctamente.
- Sistema: ponen a prueba el producto integrado y algunos de sus flujos completos.
- Aceptación: verifican expectativas relevantes para el usuario o el negocio; pueden apoyarse en el sistema funcionando de punta a punta.

Ejemplo complementario: sistema de reservas. Unitaria: validar fecha futura. Integración: verificar la escritura de una reserva en base de datos. Sistema: reservar desde interfaz hasta persistencia. Aceptación: confirmar que no sea posible reservar dos veces el mismo asiento.

11. Beneficios de la automatización (diap. 24)

La diapositiva lista seis beneficios; el alcance concreto depende de una automatización bien seleccionada y mantenida.

Rápida ejecución

Permite repetir conjuntos de pruebas en poco tiempo y acortar el feedback.

Reusabilidad

Una misma prueba puede ejecutarse frente a muchas modificaciones.

Mayor cobertura

Hace viable ejecutar más escenarios con frecuencia, aunque no garantiza cubrir todos los riesgos.

Mayor precisión

Comprueba resultados definidos de manera consistente y disminuye errores manuales repetitivos.

Mayor alcance

Puede ejecutarse en diferentes entornos, combinaciones de datos o configuraciones, si fue diseñada para ello.

Reducción de costos

Puede reducir esfuerzo repetitivo a largo plazo; hay costo inicial y mantenimiento, por lo que no todo merece automatizarse.

Aclaración complementaria: la automatización verifica lo que se le indicó comprobar. No reemplaza preguntas sobre requisitos ambiguos, exploración humana ni análisis de necesidades de negocio.